Term Project : Written Report

For this project I had to predict whether Home Depot's stock goes up or down (I picked HD because my last name is Hamra), using both market data and financial news. Here is how I approached each part, what was hard, how I solved it, and what I found interesting along the way, i also documented very well my approach on the myp python notebook file.

Question 1 : Predicting up/down from price

My first approach was to pull HD's daily data from Yahoo Finance and add the indicators asked in the question (moving averages, RSI, MACD, volatility). The first real issue I hit was that the price climbs a lot over the years, from around $20 to $300+, so if I trained on the old cheap years and tested on the recent expensive ones, the model would basically see values it had never seen before. After some research I realized I had to make the features stationary, so instead of raw dollar values I used percentage changes and ratios that stay comparable across time. The other important thing was to not shuffle the data, since it's a time series, so I split it by time order. I then built LSTM, GRU and CNN models on 30-days windows. I experimented a lot with the units, dropout and window size, but honestly everything kept landing around 0.50, which is basically random. That's when I understood it was not a model problem but a data problem: daily direction is close to a coin flip, which matches the efficient-market idea.

Question 2 : Combining the models

For question 2 I had to combine my three models into one. I looked online and found there are mainly three ways: bagging, boosting and stacking. Bagging and boosting didn't really fit my case, so I went with stacking, where a small meta-model learns to combine the predictions of the others. At first I tried adding a hidden layer to the meta-model, but it backfired and just overfit on my small data, so I kept it simple with a single sigmoid, which is basically a logistic regression. What I found interesting here was the gap between validation (approx. 0.53) and test (approx. 0.48): since they cover different time periods and the signal is near random, the number just moves around 0.5. I also learned that stacking only helps when the base models make “different” mistakes, and mine were too correlated, so combining them gave no real improvement.

Question 3 : News sentiment & stock reaction

Question 3 was the hardest, mostly because of the data. I searched a lot for clean financial news and couldn't find exactly what I wanted, so I ended up inspecting the MarketWatch page and scraping the HD headlines from 2020 to 2026 myself, then combined that with a Kaggle dataset covering 2010 to 2020. One issue was that the Kaggle ticker tags were dirty (some headlines tagged HD weren't even about Home Depot), so I filtered to keep only headlines that actually mention "Home Depot". My headlines had no sentiment labels, so I used the Financial PhraseBank dataset (already labeled) to train an LSTM and a GRU, then applied them to my own headlines, plus VADER as a rule-based baseline. I also hit a funny bug where my model "didn't learn" (AUC 0.48), and I realized I had simply never run the training cell, once trained it reached around 0.92. I handled the class imbalance

with class weights and played with the decision threshold for the negative class. The interesting part: sentiment is genuinely learnable, but when I compared the daily sentiment with the actual returns the correlation was basically zero, and predicting up/down directly from headlines was again random.

Question 4 : Transformer & BERT

For question 4 I repeated the sentiment task with a Transformer and with BERT. The main issue was technical: the TensorFlow version of BERT didn't work with my Keras 3 setup, so I switched to running BERT on PyTorch, which is the standard backend anyway. This part was the most satisfying: BERT clearly beat everything (AUC 0.99) while my from-scratch Transformer landed around the LSTM/GRU (approx. 0.90), which makes sense because Transformers need a lot of data and I didn't have much. The really interesting thing came when I validated the models on my *actual* HD headlines instead of PhraseBank: the small models dropped to around 0.70 while BERT held at 0.88. That showed me the LSTM/GRU had overfit the PhraseBank vocabulary, while BERT generalized much better thanks to pre-training. But even BERT's near-perfect sentiment still didn't help predict the returns.

Question 5 : Combining everything (fusion)

Question 5 was about combining everything into one final model: the price features from Q1 plus the outputs of all the previous models, to predict up or down. The hard part was lining up the data, because the price side has one row per trading day while the news is sparse, so I built a daily master table and filled the no-news days with a neutral value. I also had an annoying bug where all my dates showed as 1970, which happened because the dates were stored in a column and not the index, and my first fix accidentally grabbed the integer index instead, once I read the Date column directly it worked. The combined model landed around 0.48, the same as price-only and news-only, and a variance check over several seeds confirmed it was genuinely random (0.48 plus or minus 0.01), not just one unlucky run. So combining the two worlds didn't help, because both were already close to random on their own.

Final results and takeaways

Overall, the main lesson is that sentiment is learnable (BERT got close to perfect) but Home Depot's daily up/down direction is not predictable from price, news, or both combined, every model ended up at the coin-flip baseline, which is exactly what the efficient-market hypothesis predicts. For me the real value wasn't beating the market, it was building the full pipeline correctly and testing it honestly: being careful about leakage, using proper time-based splits, validating the sentiment on real HD headlines, and checking my results over several runs instead of trusting one lucky number.